Computer Vision / Video Perception Take-Home

Senior Robotics Perception Exercise

Time box: 6–8 hours of work, hard cap 10 hours.

Deadline: end of next week, Friday, June 19, 2026. Please work at a reasonable pace alongside current commitments; this is not intended as a rush exercise.

Goal: Show practical CV engineering judgment on real egocentric video: detection or segmentation, temporal consistency, evaluation, failure analysis, and one bounded iteration from feedback.

Context

You are given four egocentric videos from rights-cleared, PII-cleared household and workshop tasks. Your job is to build a compact perception pipeline that identifies relevant entities, reasons over time, and reports where the system works and fails.

This is not a benchmark contest. We care more about good technical judgment than perfect model performance. Use any reasonable open-source models, libraries, or classical CV methods. You do not need to train a model unless you believe training is the right tradeoff and can justify it.

You do not need to process all frames. You may process sampled frames or selected clips. We prefer a clearly justified subset with honest evaluation over shallow processing of all footage.

Included Files

- `videos.json`: four stratified video records with download URLs, task metadata, and segment annotations.
- `video_manifest.md`: human-readable summary of the four selected videos.
- `takehome_brief.pdf`: this brief.

Video Sample

ID	Category	Primary object	Task	Duration
`767223`	cleaning	dishes	Washing dishes in a kitchen sink under running water.	384.3s
`870855`	laundry garment care	clothes	Folding clothing from a laundry basket onto a bed.	351.5s
`165895`	food preparation	wooden spoon	Cooking stew, adding ingredients, and stirring frequently.	349.8s
`839878`	repair assembly	hairdryer	Disassembling and reassembling a hairdryer.	644.5s

Minimum Scope

Pick 2 of the 4 videos: at least one household task and one workshop or object-manipulation task. Build a compact pipeline on sampled frames or clips. Detect or segment 2–4 useful entity types, add simple

temporal association, evaluate on a small manually reviewed subset, simulate 8–12 reviewer feedback items, and show one bounded iteration.

Optional: run lightweight inference or qualitative checks on all 4 videos if time allows. Do not sacrifice the quality of the 2-video core evaluation to cover every video.

Task

Build a small perception system for your selected videos. The system should do three things:

1. Detect or segment 2–4 useful entity classes that matter for task understanding, such as hands, the primary manipulated object, a container or surface, and one ambiguous or unknown object class.
2. Track or associate entities across time well enough to reason about continuity, occlusion, re-entry, missed detections, and ID drift. Tracking can be simple. We care about whether your temporal logic is useful and inspected, not whether you implement a production tracker.
3. Evaluate the system honestly and explain what changed after one feedback pass.

Required Deliverables

Submit a zip file containing:

- `README.md`: setup, runtime, assumptions, selected videos, sampling plan, and how to reproduce your results.
- `src/` or `notebook/`: code or notebook used to run the pipeline.
- `outputs/predictions.jsonl`: one record per prediction or track segment.
- `outputs/visualizations/`: images, short clips, or notebook cells showing representative results.
- `report.md` or `report.pdf`: concise technical report, maximum 4 pages.

Prediction Format

Use a practical schema. At minimum, include:

- video ID, frame index or timestamp range
- class label
- box and/or mask reference
- track ID if available
- confidence score if available
- model or method used
- notes for uncertainty or known failure modes

Feedback Loop

Create a small simulated reviewer-feedback pass. You can define 8–12 feedback items yourself after inspecting the model output. Examples:

- missed hand under occlusion
- object identity switch
- track fragmentation after fast motion
- false positive on background clutter
- poor mask boundary around transparent or deformable objects

Then update the system in a bounded way: threshold tuning, prompt changes, simple post-processing, better frame sampling, tracking logic, or a small fine-tuning experiment. Show before/after evidence and explain what improved or did not improve.

Evaluation

Use metrics that match what you built. Reasonable choices include:

- detection precision, recall, AP, or IoU on manually checked frames
- segmentation IoU or Dice on manually checked frames
- track continuity, ID switches, track fragmentation, or re-identification failures
- correction capture rate: fraction of feedback items fixed or reduced
- qualitative failure taxonomy with concrete examples

If you create your own small labeled subset, describe how many frames you labeled and why those frames are representative.

Allowed Tools

Use any stack you think is appropriate. Examples include:

- SAM, SAM2, Mask R-CNN, YOLO, RT-DETR, D-FINE, Grounding DINO, or classical CV methods
- ByteTrack, CoTracker, optical flow, Kalman filtering, or simple association heuristics
- PyTorch, OpenCV, torchvision, Hugging Face Transformers, Ultralytics, Detectron2, or equivalent

What We Will Look For

Area	Signal	Weight
CV fundamentals	Sound model/method selection, correct preprocessing, useful detection or segmentation output.	25%
Temporal reasoning	Tracks or temporal logic are useful, not just frame-by-frame screenshots.	20%
Evaluation rigor	Metrics and failure analysis are honest, specific, and tied to what the system does.	20%
Feedback-driven iteration	Reviewer feedback changes the system or the analysis in a measurable way.	15%
Engineering quality	Reproducible, readable, bounded runtime, clean output structure.	10%

Technical judgment	Clear tradeoffs, knows when not to train, sensible next steps.	10%

Strong Signals

- You make a simple system reliable before making it complex.
- You inspect failures rather than hiding them.
- You choose metrics that match the actual task.
- You explain model limitations under occlusion, motion blur, clutter, deformation, and object re-entry.
- You show how human feedback would improve the next iteration.

Anti-Patterns

- A polished demo with no evaluation.
- Treating model predictions as ground truth.
- Training without first auditing the data and labels.
- No temporal reasoning.
- No clear failure examples.
- A large, fragile pipeline that cannot be reproduced.

Optional Extensions

If you have extra time, choose one:

- fine-tune a small detector or segmenter on a tiny labeled subset and compare against the baseline
- export a model to ONNX and discuss expected TensorRT or Triton deployment constraints
- add embedding-based retrieval for visually similar failure cases
- design a one-week plan for collecting 500 labels and improving the system

Follow-Up Discussion

Be prepared to present your approach for 20–30 minutes and discuss tradeoffs, failures, and next steps. We will care more about how you reason about the system than whether every edge case is solved. Keep the final report concise: we want to understand how you think, what you built, how you measured it, and what you would do next.